

Oracle SQL & PL/SQL Commands Reference

Supply Chain Scorer & Risk Management Database System

Reference Sheet: 15 SQL Queries (DML/DQL/Joins/Aggregations) + 5 PL/SQL Database Objects (Block/Func/Proc/Trig/Pkg)

Oracle 19c/21c Syntax | Tables: SELLER, PRODUCT, PURCHASE_ORDER, PO_LINE, DELIVERY, QUALITY_INSPECTION, RISK_SCORE

PART 1: 15 SQL COMMANDS (DML & DQL)

Commands 01-05: Single-Row Inserts, Relative Date Arithmetic, Foreign Keys, Sequence Generators

SQL 01 INSERT into SELLER Table

DML / Single Insert

Purpose: Inserts a new vendor/seller record with ID, Zip Code, City, and State into the SELLER master table.

```
INSERT INTO SELLER (SellerID, SellerZipCodePrefix, SellerCity, SellerState)
VALUES ('S999', '12345', 'New City', 'SP');
```

SQL 02 INSERT into PRODUCT Table

DML / Single Insert

Purpose: Inserts a new product catalog item with Category into the PRODUCT master table.

```
INSERT INTO PRODUCT (ProductID, ProductCategoryName)
VALUES ('P999', 'Electronics');
```

SQL 03 INSERT into PURCHASE_ORDER Table

DML / Date Arithmetic

Purpose: Creates a purchase order header with relative date calculation (SYSDATE - 10, SYSDATE + 5) and status.

```
INSERT INTO PURCHASE_ORDER (PO_ID, OrderDate, ExpectedDeliveryDate, OrderStatus)
VALUES ('P0999', SYSDATE - 10, SYSDATE + 5, 'delivered');
```

SQL 04 INSERT into PO_LINE Table

DML / FK Relationship

Purpose: Inserts an order line item linking PO_ID, ProductID, and SellerID with price, freight, and shipping deadline.

```
INSERT INTO PO_LINE (PO_ID, LineNo, ProductID, SellerID, UnitPriceAtOrder, FreightValue, ShippingLimitDate)
VALUES ('P0999', 1, 'P999', 'S999', 150.00, 20.00, SYSDATE - 8);
```

SQL 05 INSERT into DELIVERY Table

DML / Sequence Generator

Purpose: Records carrier delivery metrics and computed delay days using sequence DELIVERY_SEQ.NEXTVAL.

```
INSERT INTO DELIVERY (DeliveryID, PO_ID, DeliveredCarrierDate, ActualDeliveryDate,
                      SellerDispatchDelayDays, CarrierTransitDelayDays, DelayDays)
VALUES (DELIVERY_SEQ.NEXTVAL, 'P0999', SYSDATE - 7, SYSDATE - 2, 1.5, 5.0, 0);
```

PART 1: 15 SQL COMMANDS (CONTINUED)

Commands 06-10: Inspection Records, Row Updates, Safe Record Deletions, and Filtered Selects

SQL 06**INSERT into QUALITY_INSPECTION Table**

DML / Sequence & Nulls

Purpose: Records customer review score, rejection status, review timestamp, and comments using INSPECTION_SEQ.

```
INSERT INTO QUALITY_INSPECTION (InspectionID, PO_ID, InspectionDate, ReviewScore, RejectionFlag,
                                ReviewAnswerTimestamp, ReviewCommentTitle, ReviewCommentMessage, DefectTypeID)
VALUES (INSPECTION_SEQ.NEXTVAL, 'P0999', SYSDATE - 1, 5, 'No',
        SYSDATE, 'Great!', 'Arrived on time and in perfect condition.', NULL);
```

SQL 07**UPDATE DELIVERY Record**

DML / Row Update

Purpose: Modifies calculated transit delay days for a specific purchase order delivery.

```
UPDATE DELIVERY
SET DelayDays = 2
WHERE PO_ID = 'P0999';
```

SQL 08**UPDATE SELLER Location**

DML / Row Update

Purpose: Updates the registered city location for a specific seller record.

```
UPDATE SELLER
SET SellerCity = 'Sao Paulo'
WHERE SellerID = 'S999';
```

SQL 09**DELETE FROM SELLER Table**

DML / Safe Row Deletion

Purpose: Demonstrates inserting and safely removing a dummy test seller without violating relational integrity.

```
-- 1. Insert temporary dummy seller record
INSERT INTO SELLER (SellerID, SellerZipCodePrefix, SellerCity, SellerState)
VALUES ('S_DEL', '00000', 'Delete City', 'XX');

-- 2. Safely delete the dummy record
DELETE FROM SELLER WHERE SellerID = 'S_DEL';
```

SQL 10**SELECT with WHERE Clause**

DQL / Equality Filter

Purpose: Retrieves all seller records operating from the city of Sao Paulo.

```
SELECT *
FROM SELLER
WHERE SellerCity = 'Sao Paulo';
```

PART 1: 15 SQL COMMANDS (CONTINUED)

Commands 11-15: Group By Aggregations, Multi-Table Joins, Sorting, and Left Outer Joins

SQL 11 SELECT with GROUP BY and COUNT()

DQL / Aggregation

Purpose: Aggregates and counts the total number of registered sellers grouped by state.

```
SELECT SellerState, COUNT(*) AS TotalSellers
FROM SELLER
GROUP BY SellerState;
```

SQL 12 SELECT with 3-Table INNER JOIN

DQL / Multi-Table Join

Purpose: Joins PO_LINE, PRODUCT, and PURCHASE_ORDER to retrieve order numbers alongside product category details.

```
SELECT po.PO_ID, p.ProductID, p.ProductCategoryName
FROM PO_LINE pl
JOIN PRODUCT p ON pl.ProductID = p.ProductID
JOIN PURCHASE_ORDER po ON pl.PO_ID = po.PO_ID;
```

SQL 13 SELECT with JOIN, GROUP BY, and AVG()

DQL / Analytical Query

Purpose: Calculates average delivery delay in days per seller across SELLER, PO_LINE, and DELIVERY tables.

```
SELECT s.SellerID, AVG(d.DelayDays) AS AvgDelay
FROM SELLER s
JOIN PO_LINE pl ON s.SellerID = pl.SellerID
JOIN DELIVERY d ON pl.PO_ID = d.PO_ID
GROUP BY s.SellerID;
```

SQL 14 SELECT with Filtering and ORDER BY DESC

DQL / Filter & Sort

Purpose: Retrieves low quality inspection scores (ReviewScore <= 3) sorted with the newest inspections first.

```
SELECT *
FROM QUALITY_INSPECTION
WHERE ReviewScore <= 3
ORDER BY InspectionDate DESC;
```

SQL 15 SELECT with LEFT OUTER JOIN

DQL / Outer Join

Purpose: Retrieves all sellers along with risk scores, retaining sellers who have not yet undergone risk evaluation.

```
SELECT s.SellerID, s.SellerCity, rs.WeightedRiskScore
FROM SELLER s
LEFT JOIN RISK_SCORE rs ON s.SellerID = rs.SellerID;
```

PART 2: 5 PL/SQL COMMANDS (PROCEDURAL OBJECTS)

Objects 01-04: Anonymous Blocks, Stored Functions, Stored Procedures, and Row-Level Database Triggers

PL/SQL 01**Anonymous Block with Exception Handling**

Procedural / Anonymous Block

Purpose: Queries the maximum delivery delay from DELIVERY table and prints output to DBMS console with error handling.

```
SET SERVEROUTPUT ON;
DECLARE
    v_max_delay NUMBER;
BEGIN
    SELECT MAX(DelayDays) INTO v_max_delay FROM DELIVERY;
    DBMS_OUTPUT.PUT_LINE('Maximum Delivery Delay is: ' || NVL(v_max_delay, 0) || ' days.');
```

EXCEPTION

```
    WHEN NO_DATA_FOUND THEN
        DBMS_OUTPUT.PUT_LINE('No delivery data found.');
```

END;
/

PL/SQL 02**Stored Function: get_product_category**

Stored Function / Return Value

Purpose: Accepts a Product ID, returns category name from PRODUCT table, and handles NO_DATA_FOUND exception.

```
CREATE OR REPLACE FUNCTION get_product_category (p_product_id IN VARCHAR2)
RETURN VARCHAR2
IS
    v_category VARCHAR2(100);
BEGIN
    SELECT ProductCategoryName INTO v_category
    FROM PRODUCT
    WHERE ProductID = p_product_id;

    RETURN v_category;
EXCEPTION
    WHEN NO_DATA_FOUND THEN
        RETURN 'Unknown Category';
END;
```

/

PL/SQL 03**Stored Procedure: add_new_seller**

Stored Procedure / Autonomous DML

Purpose: Inserts a new seller with input parameters, commits the transaction, and handles duplicate key errors.

```
CREATE OR REPLACE PROCEDURE add_new_seller (
    p_seller_id IN VARCHAR2,
    p_zip_code  IN VARCHAR2,
    p_city      IN VARCHAR2,
    p_state     IN VARCHAR2
)
IS
BEGIN
    INSERT INTO SELLER (SellerID, SellerZipCodePrefix, SellerCity, SellerState)
    VALUES (p_seller_id, p_zip_code, p_city, p_state);
    COMMIT;
    DBMS_OUTPUT.PUT_LINE('Seller ' || p_seller_id || ' added successfully.');
```

EXCEPTION

```
    WHEN DUP_VAL_ON_INDEX THEN
        DBMS_OUTPUT.PUT_LINE('Seller ' || p_seller_id || ' already exists.');
```

END;
/

PL/SQL 04**Database Trigger: trg_po_order_date**

Database Trigger / BEFORE INSERT

Purpose: Row-level trigger on PURCHASE_ORDER that automatically defaults OrderDate to current SYSDATE if null.

```
CREATE OR REPLACE TRIGGER trg_po_order_date
BEFORE INSERT ON PURCHASE_ORDER
FOR EACH ROW
BEGIN
    IF :NEW.OrderDate IS NULL THEN
        :NEW.OrderDate := SYSDATE;
    END IF;
END;
```

/

PART 2: 5 PL/SQL COMMANDS (CONTINUED)

Object 05: Database Package Specification & Body Encapsulating Risk Evaluation Logic

PL/SQL 05**Database Package: risk_mgmt_pkg (Spec & Body)**

Database Package / Modular Encapsulation

Purpose: Encapsulates risk management business logic: procedure evaluate_seller_risk and function get_seller_risk_score.

```

-- =====
-- 1. PACKAGE SPECIFICATION (INTERFACE DECLARATION)
-- =====
CREATE OR REPLACE PACKAGE risk_mgmt_pkg AS
    PROCEDURE evaluate_seller_risk(p_seller_id VARCHAR2, p_eval_period VARCHAR2);
    FUNCTION get_seller_risk_score(p_seller_id VARCHAR2, p_eval_period VARCHAR2) RETURN NUMBER;
END risk_mgmt_pkg;
/

-- =====
-- 2. PACKAGE BODY (IMPLEMENTATION LOGIC)
-- =====
CREATE OR REPLACE PACKAGE BODY risk_mgmt_pkg AS
    PROCEDURE evaluate_seller_risk(p_seller_id VARCHAR2, p_eval_period VARCHAR2) IS
        v_dummy NUMBER;
    BEGIN
        -- Simplified logic for demonstration
        SELECT COUNT(*) INTO v_dummy FROM SELLER WHERE SellerID = p_seller_id;
        IF v_dummy > 0 THEN
            DBMS_OUTPUT.PUT_LINE('Evaluated risk for seller ' || p_seller_id || ' for period ' || p_eval_period);
        END IF;
    END evaluate_seller_risk;

    FUNCTION get_seller_risk_score(p_seller_id VARCHAR2, p_eval_period VARCHAR2) RETURN NUMBER IS
        v_score NUMBER(5,2);
    BEGIN
        SELECT WeightedRiskScore INTO v_score
        FROM RISK_SCORE
        WHERE SellerID = p_seller_id AND EvaluationPeriod = p_eval_period;
        RETURN v_score;
    EXCEPTION
        WHEN NO_DATA_FOUND THEN
            RETURN -1; -- indicates no score found
    END get_seller_risk_score;
END risk_mgmt_pkg;
/

```

EXECUTION & DATABASE LAB VERIFICATION NOTES:

- Sequences Required: Ensure sequence objects DELIVERY_SEQ and INSPECTION_SEQ are created prior to executing SQL 05 and SQL 06.
- Transaction Control: Execute 'COMMIT;' following DML operations to persist changes permanently to the database.
- PL/SQL Console Output: Ensure 'SET SERVEROUTPUT ON;' is executed in SQL*Plus or Oracle SQL Developer prior to running PL/SQL blocks.
- Integrity Constraints: SQL 09 demonstrates a safe delete using a temporary seller record ('S_DEL') to prevent foreign key constraint violations.